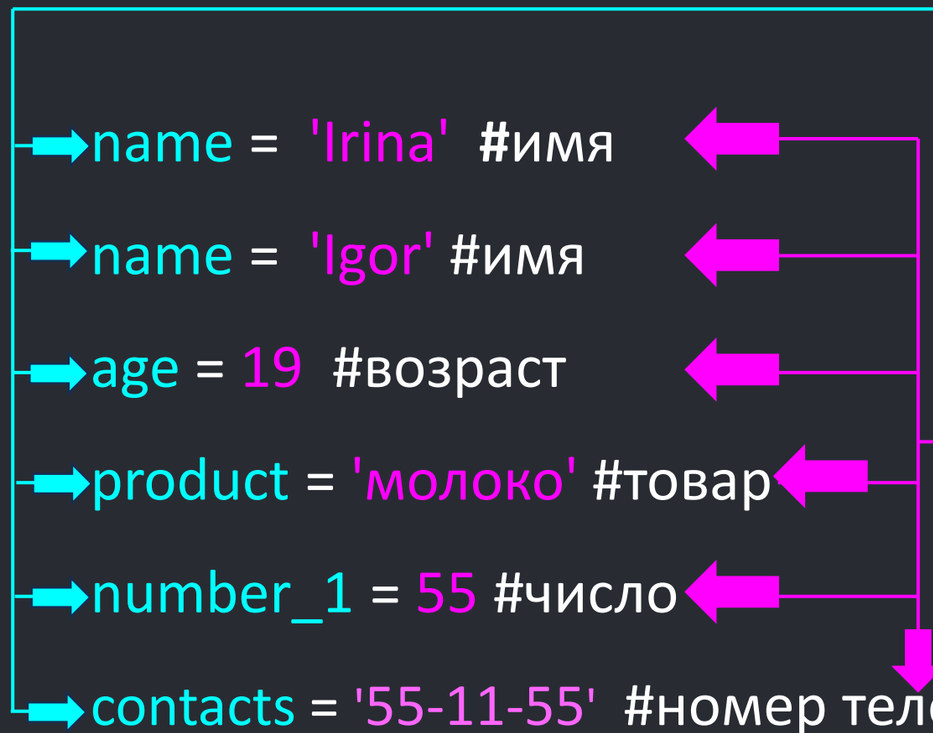
Введение в типы данных в Python

Python – это мощный язык программирования, который предоставляет широкий спектр типов данных для хранения и обработки различной информации. В этом введении мы рассмотрим основные типы переменных, такие как целые числа, числа с плавающей точкой, строки и логические значения.

СЦ **Цехош София Ивановна**

Переменные



Переменная - это именованный контейнер для заданного значения.

- ✓ Переменную желательно называть на английском языке.
- ✓ Нейминг (название переменной) должен отражать суть переменной (желательно называть существительным и с маленькой буквы).

Строковые переменные необходимо называть с маленькой буквы.
#С большой буквы называют классы.

Название переменных не могут **начинаться** с цифры! ➡ ~~1~~_number = 55

- строчный комментарий, который не выводится при запуске кода

❌ КАК НЕЛЬЗЯ называть переменные

1. Не использовать русский язык

```
# НЕПРАВИЛЬНО:

клиент = "Петров"  # Может вызвать ошибки!

номер_договора = 123

# ПРАВИЛЬНО:

client = "Петров"

contract_number = 123
```

❌ КАК НЕЛЬЗЯ называть переменные

2. Не использовать спецсимволы и пробелы

```
# НЕПРАВИЛЬНО:

client-name = "Иванов"  # дефис

client name = "Иванов"  # пробел

client@email = "test@mail.ru"  # спецсимвол

2024case = "дело №1"  # начинается с цифры


# ПРАВИЛЬНО:

client_name = "Иванов"

client_email = "test@mail.ru"

case_2024 = "дело №1"
```

❌ КАК НЕЛЬЗЯ называть переменные

3. Не использовать зарезервированные слова Python

```
# НЕПРАВИЛЬНО (вызовет ошибку):
class = "гражданский"  # class - зарезервированное слово
def = "определение"    # def - зарезервированное слово
import = "импорт"      # import - зарезервированное слово

# ПРАВИЛЬНО:
case_class = "гражданский"
definition = "определение"
import_type = "импорт"
```

❌ КАК НЕЛЬЗЯ называть переменные

4. Не использовать слишком общие названия

НЕПРАВИЛЬНО (непонятно, что хранится):

a = "договор купли-продажи"

b = "Московский городской суд"

x = "15.07.2024"

ПРАВИЛЬНО:

contract_type = "договор купли-продажи"

court_name = "Московский городской суд"

hearing_date = "15.07.2024"

❌ КАК НЕЛЬЗЯ называть переменные

5. Не использовать транслит (русские слова английскими буквами)

```
# НЕПРАВИЛЬНО:
dogovor = "договор №1"  # транслит
sud = "арбитражный суд"  # транслит

# ПРАВИЛЬНО:
contract = "договор №1"
court = "арбитражный суд"
```

💡 ПРАВИЛА ДЛЯ ЗАПОМИНАНИЯ:

Только английский - называем переменные по-английски

Змеиный регистр - client_name (слова через подчеркивание)

Осмысленные названия - имя должно отражать содержание

Начинать с буквы - первым символом должна быть буква

Без пробелов и спецсимволов - только буквы, цифры и _

Запомните: хорошо названная переменная - это уже половина понимания кода!

Использование кавычек в Python

Python предлагает гибкость в использовании одинарных и двойных кавычек для определения строковых значений.

Одинарные кавычки

Используются для обычных строковых литералов.

```
text1 = 'Простой текст'
print(text1)
```

Вывод:

```
Простой текст
```

Двойные кавычки

Также применяются для создания строковых значений.

```
text2 = "Текст в двойных кавычках"
print(text2)
```

Вывод:

```
Текст в двойных кавычках
```

Оба типа кавычек работают идентично, но каждый имеет свои преимущества в определённых ситуациях, например, когда необходимо вставить кавычки внутри строки, избегая экранирования:

```
text = 'Он сказал: "Привет!"' # Одинарные кавычки для текста с двойными кавычками внутри
```

Вывод:

```
Он сказал: "Привет!"
```

Комментарии в Python

Комментарии помогают объяснить код, делая его более понятным для других разработчиков и для вас самих в будущем. В Python есть два основных типа комментариев:

Однострочные комментарии

Используются для кратких пояснений в одной строке, часто после участка кода или для отключения строки кода.

```
print("Hello") # Комментарий после кода

Вывод:

Hello
```

Многострочные комментарии

Обычно применяются для документирования функций, классов или целых модулей, занимая несколько строк. В Python это строковые литералы в тройных кавычках.

```
"""Это многострочный комментарий.Он
может занимать несколько строк.Обычно
используется для документации."""

print("Код после комментария")

Вывод:

Код после комментария
```

Типы данных:

```
# str (строки)
name = 'Ирина'

# int (целые числа)
age = 35

# float (дробные числа)
fraction = 5.5

# bool (правда или ложь)
study = True # всего два значения: True и False
```

```
age = '22' # str -> '22'
age = int(age) # int -> 22
age = float(age) # float -> 22.0
age = bool(age) # bool -> True
```

Ввод и вывод данных

СЦ Цехош София Ивановна

Функция print()



print() ***выводит*** **данные** на экран

' ' – **str**



Пример:

```
print('Hello, world!')

# Вывод
Hello, world!
```

В скобках записываются переданные в функцию **параметры.**

print() ***выводит*** **данные** на экран

Функция print()



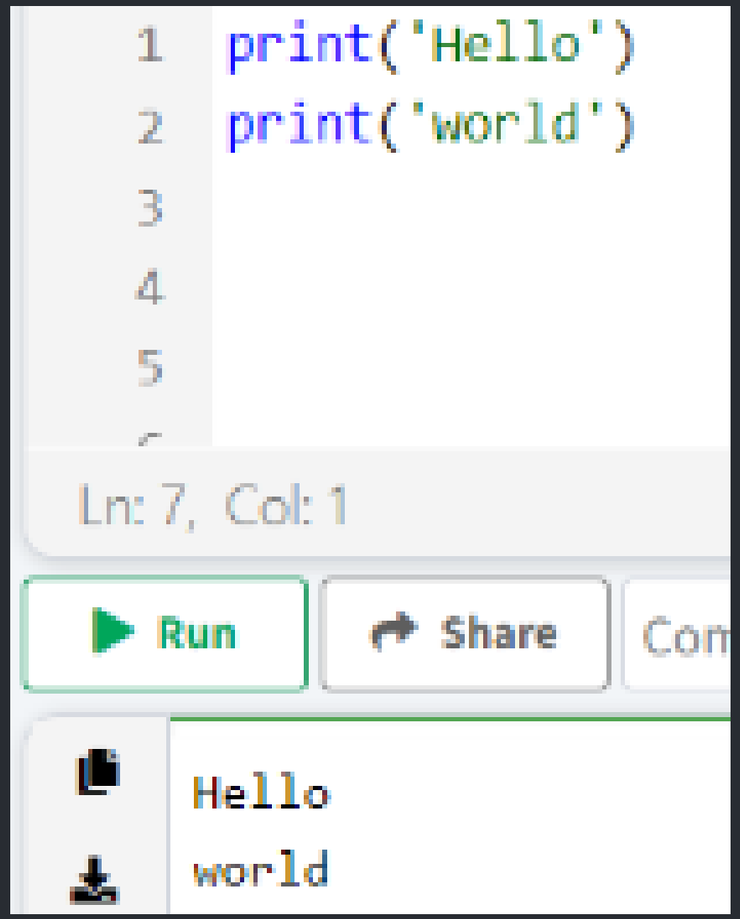
name – строковая переменная

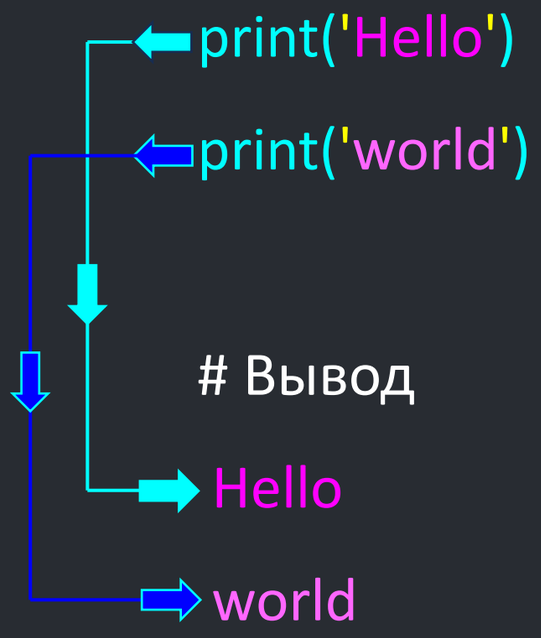
' ' – **str** (строковый тип данных)



print() ***выводит*** **данные** на экран

Функция print()





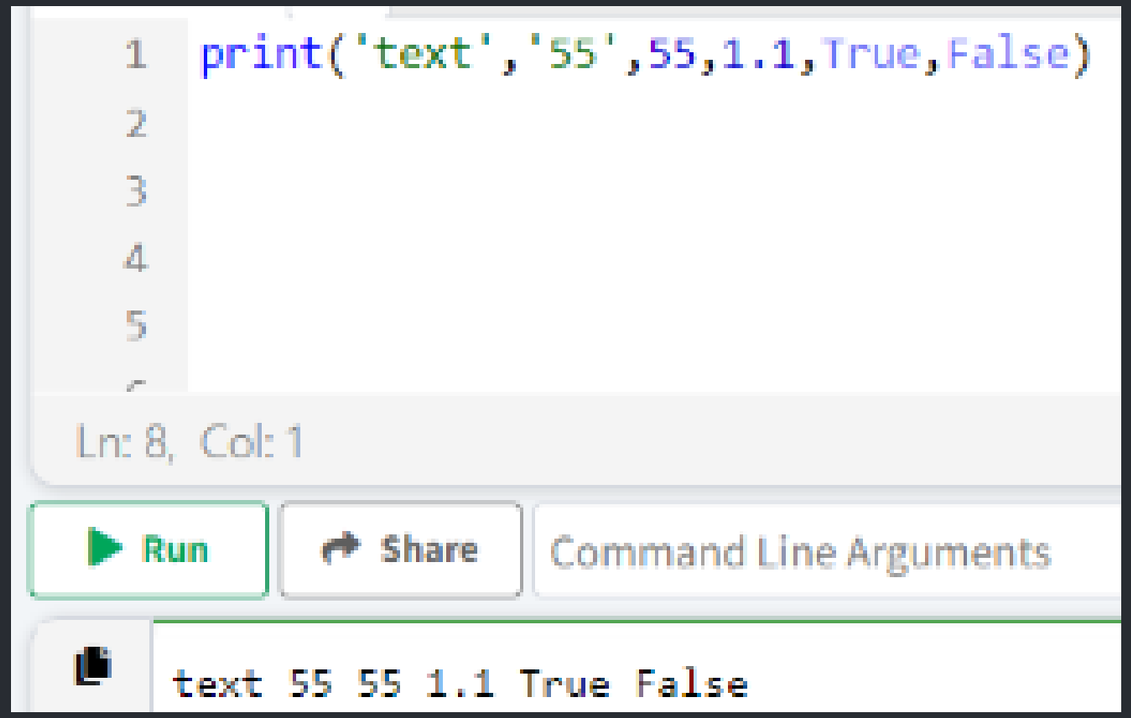
***Каждый* print()** выводит данные на **новой строке**.

print() ***выводит*** **данные** на экран

Функция print()

```
print('text','55',55,1.1,True,False)
```

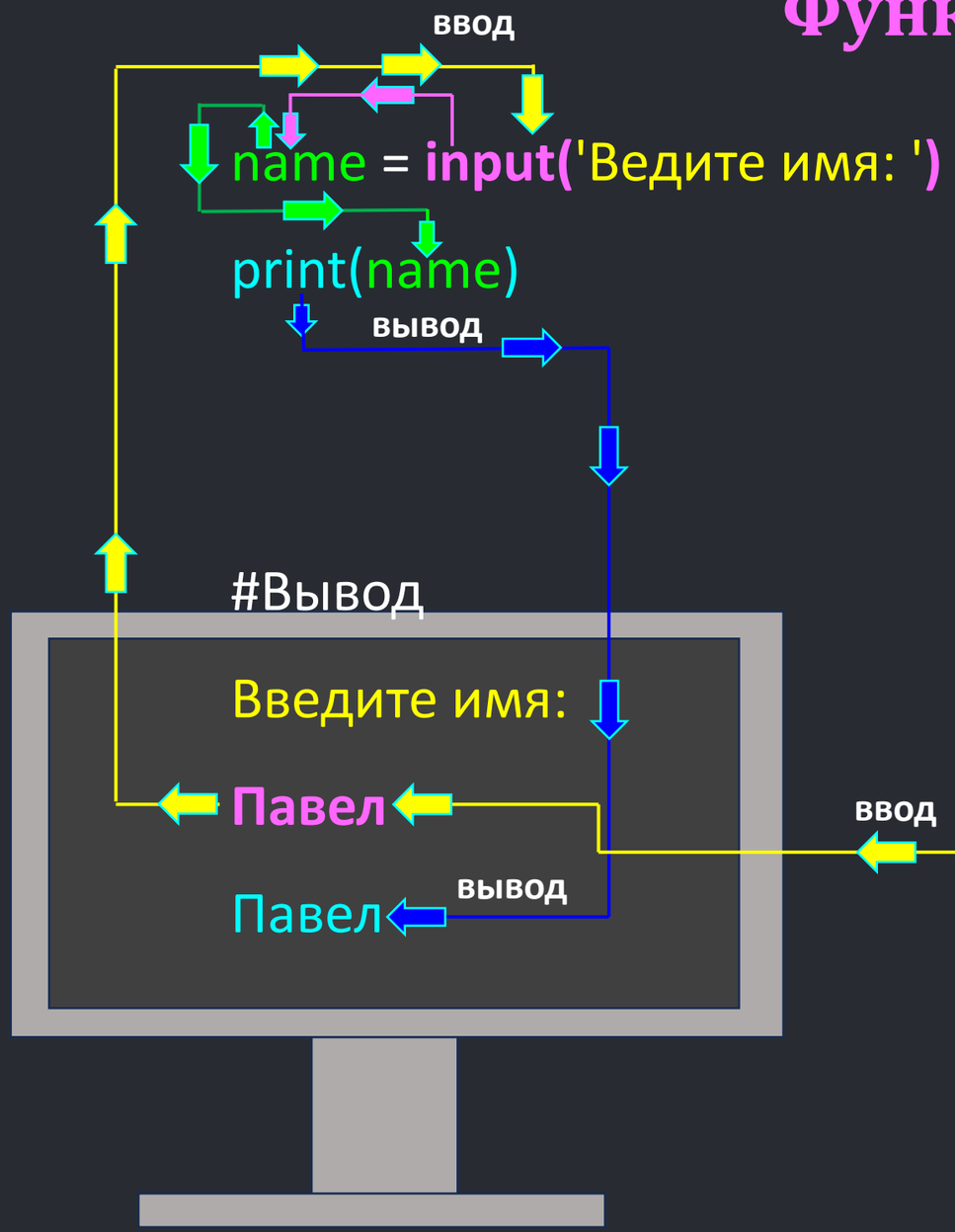
```
# Вывод
text 55 55 1.1 True False
```



Функция **print()** может принимать **несколько** входных **параметров**.

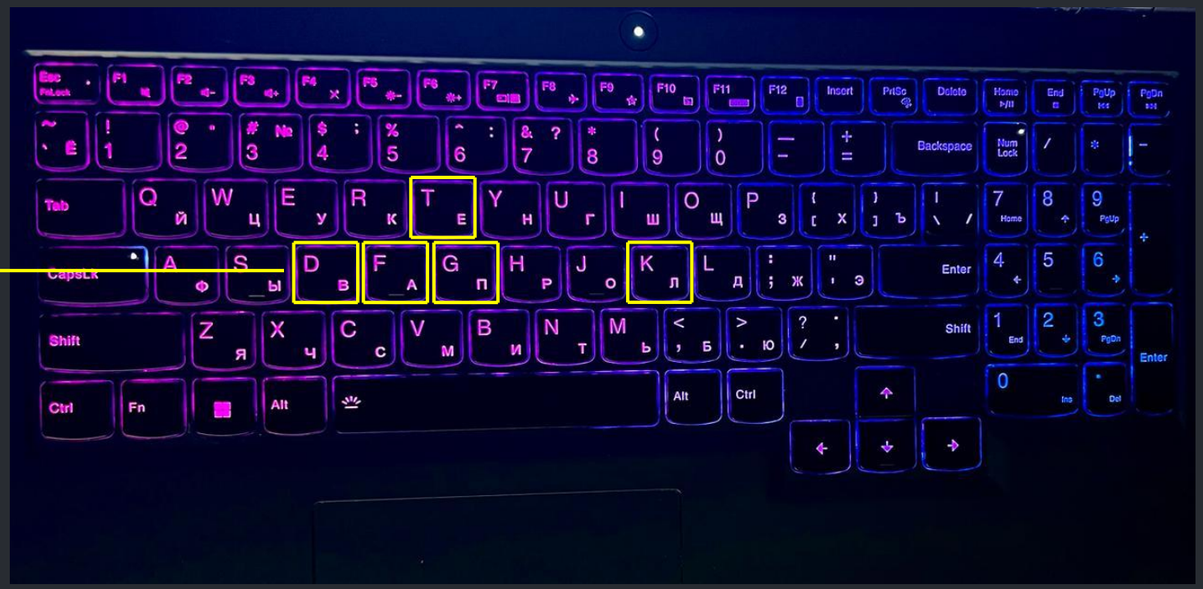
print() ***выводит*** **данные** на экран

Функция input()



Функция **input()** принимает пользовательский **ввод данных**.

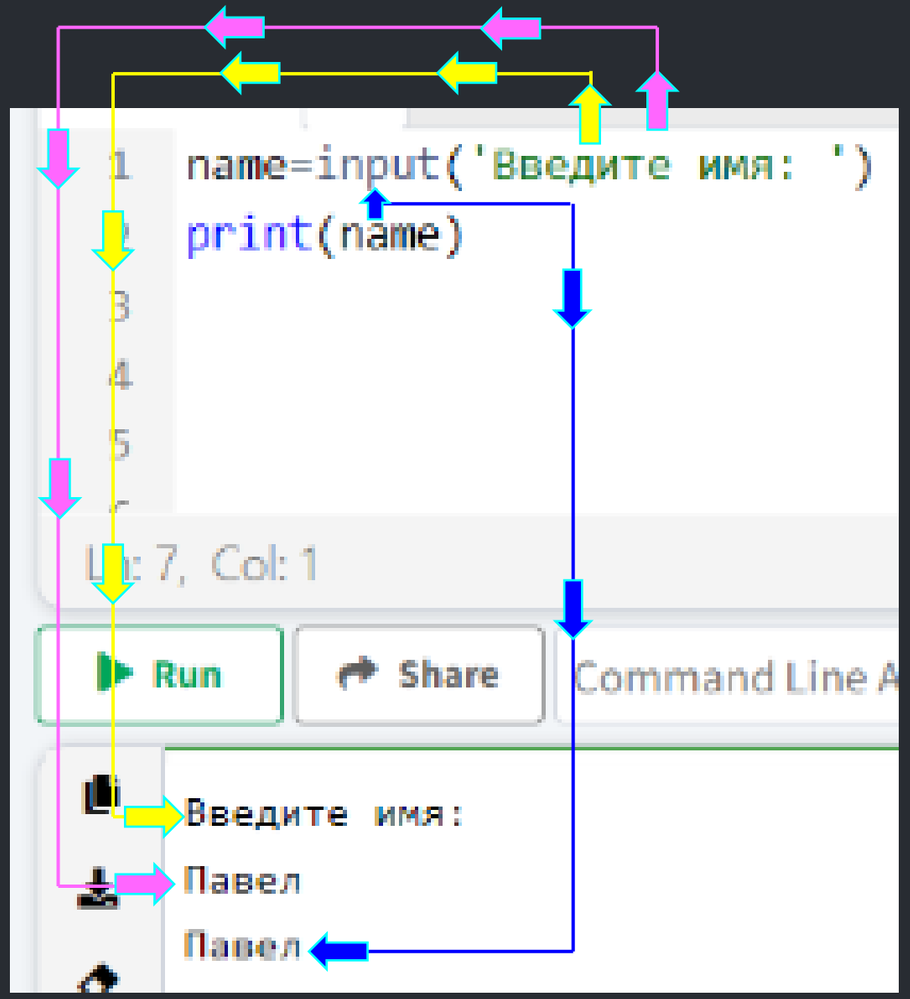
- ✓ Данные, полученные с помощью функции **input()**, имеют **строковый тип данных (str).**
- ✓ Может принимать всего лишь **один аргумент** - строку, которая выведется перед входной строкой.



Необходимо осуществить ввод данных

СЦ Цехош София Ивановна

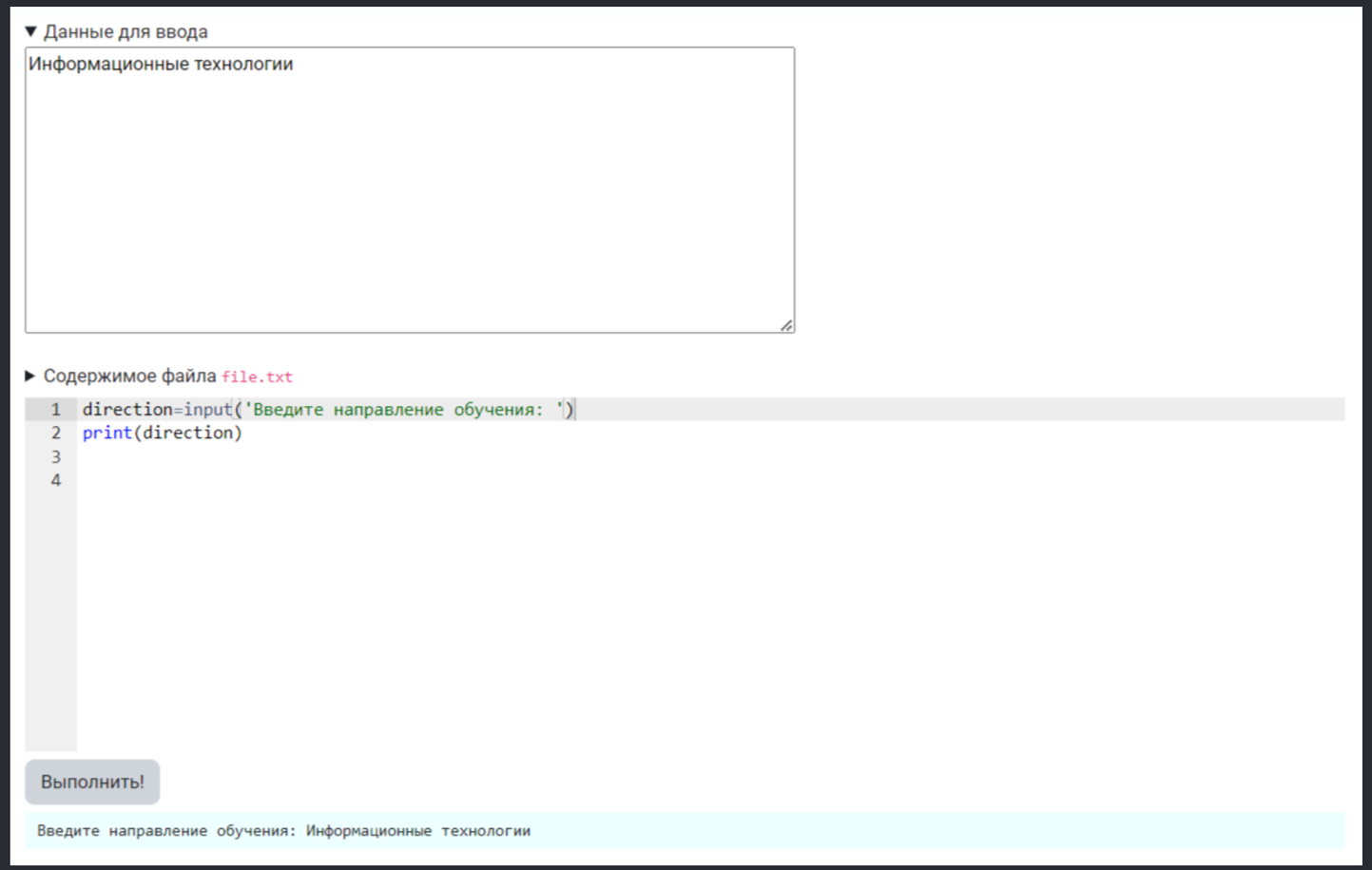
Функция input()





Функция **input()** принимает пользовательский **ввод данных**

```
direction = input('Введите направление обучения: ')
print(direction)
```



Данные, полученные с помощью функции **input()**, имеют **строковый тип данных (str).**

Сумма двух строчных чисел

```
number1 = input('Введите первое число: ')
number2 = input('Введите второе число: ')
print(number1 + number2)
```

```
# Ввод:
>> 5 #строковый тип данных
>> 4 #строковый тип данных
# Вывод:
>> 54 #конкатенация строковых данных
```

Строки можно складывать друг с другом, такое **сложение** называется их **конкатенацией или объединением.**

Функция **input()** принимает пользовательский **ввод данных**